

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)
«МАИ»

Лабораторная работа №1
по дисциплине «Программирование»
на тему:
«Двумерные массивы»
Вариант №8

Выполнили:
студенты гр. М4О-208Б-23
Порошин Г.А.
Прошин Е.С.
Проверили:
Чечиков Ю.Б.
Секретарев В.Е.

Г.Москва
2025

Содержание

Постановка задачи	2
Блок-схема	3
Описание функций	4
Листинг рабочей программы	11
Тесты	16
Некорректные тесты	16
Корректные тесты	18
Вывод	21

Постановка задачи

Кафедра 304

Курс: ИНФОРМАТИКА II семестр

Задание 4: *Двумерные массивы*

ВАРИАНТ № 8

Дана целочисленная квадратная матрица. Определить:

1. Произведение положительных элементов матрицы, которые расположены над побочной диагональю.

2. Максимум среди сумм по строкам нечетных элементов матрицы.

Используя универсальные для различных наборов исходных данных подпрограммы реализовать данный алгоритм для заданных матриц: $A(N,N)$, $B(M,M)$.

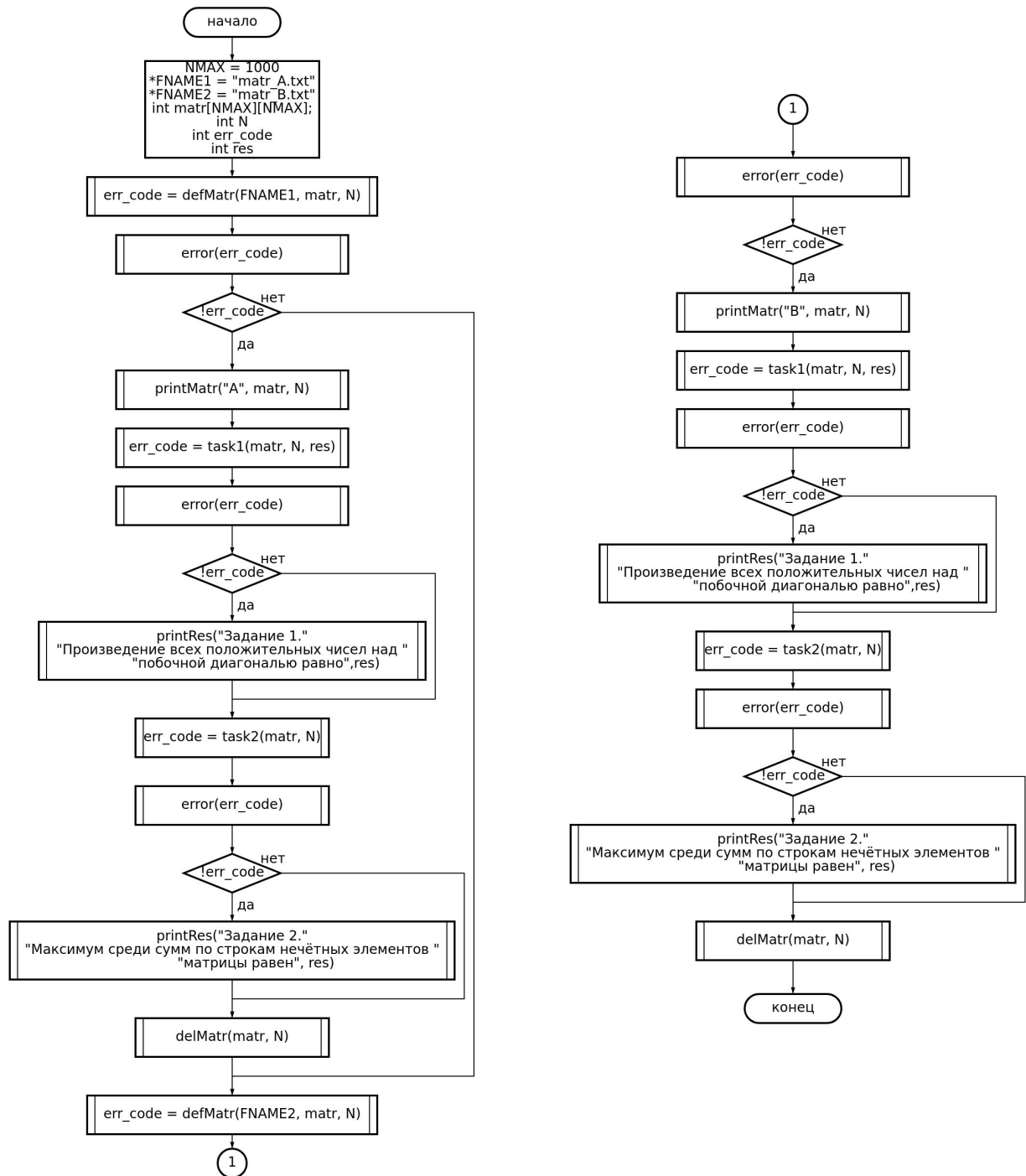
В качестве одного из вариантов исходных данных принять: $N=7$, $M=9$.

Чтение данных их файла производить с использованием функций ввода/вывода языка C++.

Алгоритм должен быть параметризован; обмен данными с подпрограммой должен осуществляться только через параметры; каждый из наборов исходных данных хранится в отдельном файле.

Реализовать программу в двух вариантах: в первом – при обращении к элементам массива использовать индексы, во втором – работать с динамическим массивом через указатели.

Блок-схема



Описание функций

- error - функция обработки ошибок

1. Прототип:

```
void error(int code);
```

2. Блок-схема:



3. Описание параметров:

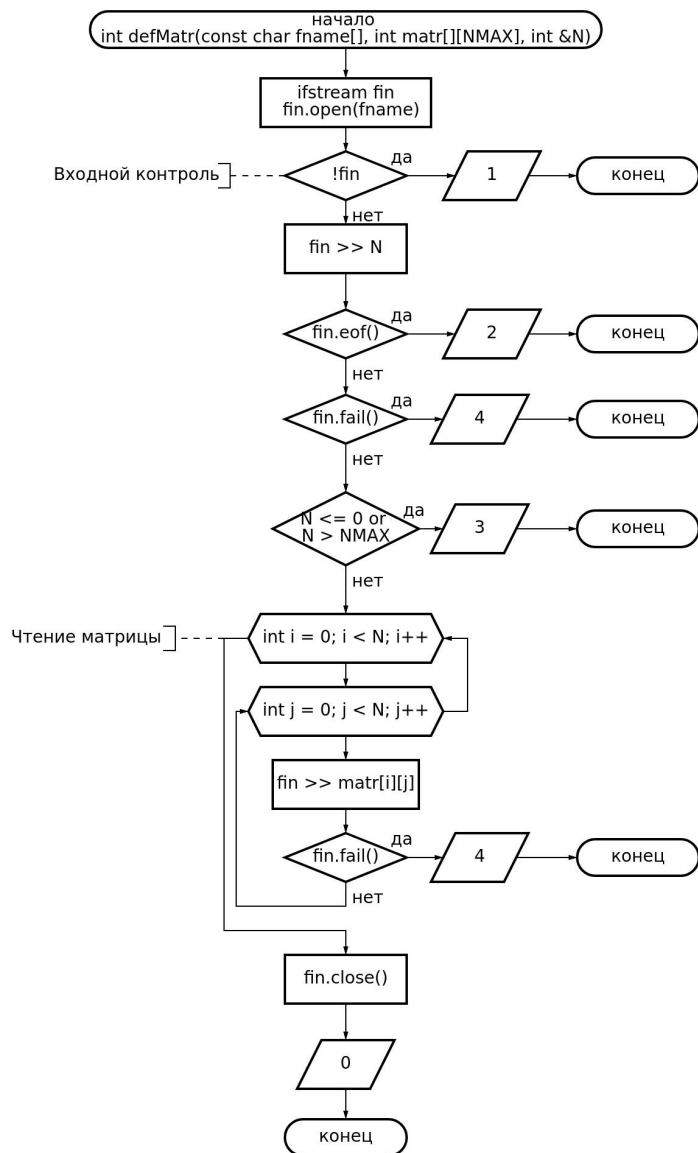
Имя	Тип	Назначение	Вх/Вых
code	int	код ошибки	Вх

- defMatr - функция задания матрицы из файла

1. Прототип:

```
int defMatr(const char fname[], int matr[][NMAX], int &N);
```

2. Блок-схема:



3. Описание параметров:

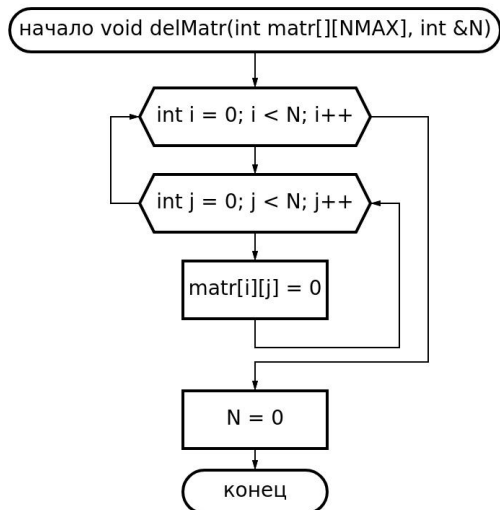
Имя	Тип	Назначение	Вх/Вых
fname	const char*	имя файла	Вх
matr	int**	матрица	Вх/Вых
N	&int	размер матрицы	Вх/Вых
code	int	код ошибки	Вых

- `delMatr` - функция очистки матрицы

1. Прототип:

```
delMatr(int matr[][NMAX], int &N);
```

2. Блок-схема:



3. Описание параметров:

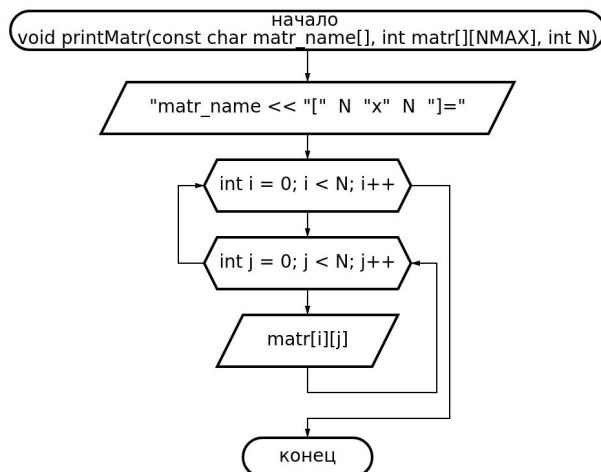
Имя	Тип	Назначение	Вх/Вых
matr	int**	матрица	Вх/Вых
N	&int	размер матрицы	Вх/Вых

- printMatr - функция вывода матрицы

1. Прототип:

```
void printMatr(const char matr_name[], int matr[][NMAX], int N);
```

2. Блок-схема:



3. Описание параметров:

Имя	Тип	Назначение	Вх/Вых
matr_name	const char*	имя матрицы	Вх

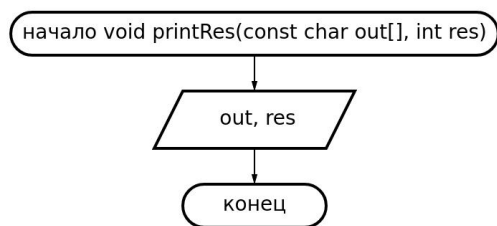
matr	int**	матрица	Вх
N	int	размер матрицы	Вх

- printRes - функция вывода результата

1. Прототип:

```
void printRes(const char out[], int res);
```

2. Блок-схема:



3. Описание параметров:

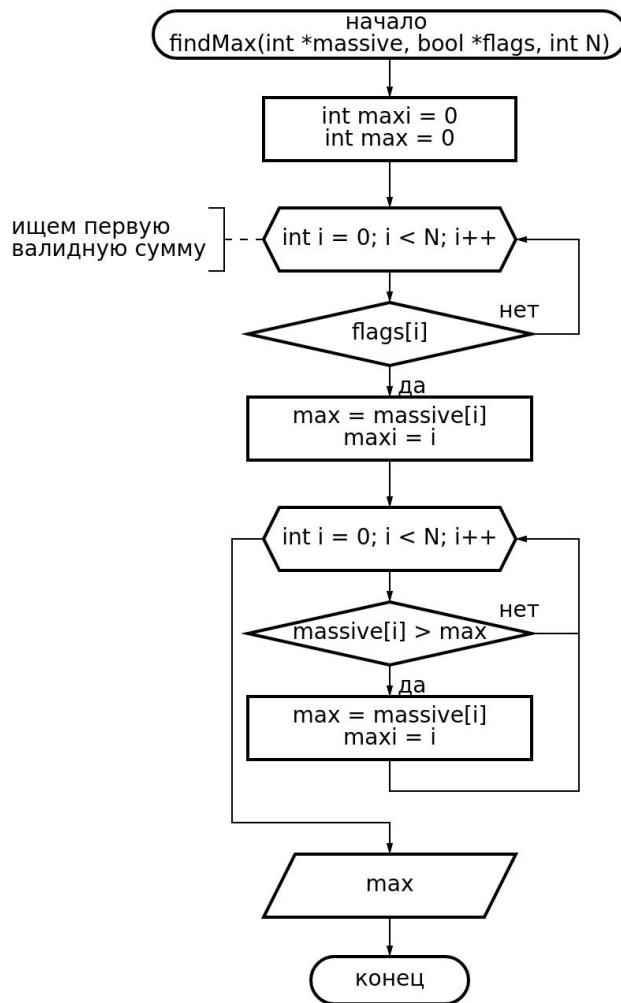
Имя	Тип	Назначение	Вх/Вых
out	const char*	текст	Вх
res	int	результат	Вх

- findMax - функция поиска максимума в векторе по вектору флагов

4. Прототип:

```
int findMax(int *massive, bool *flags, int N);
```

5. Блок-схема:



6. Описание параметров:

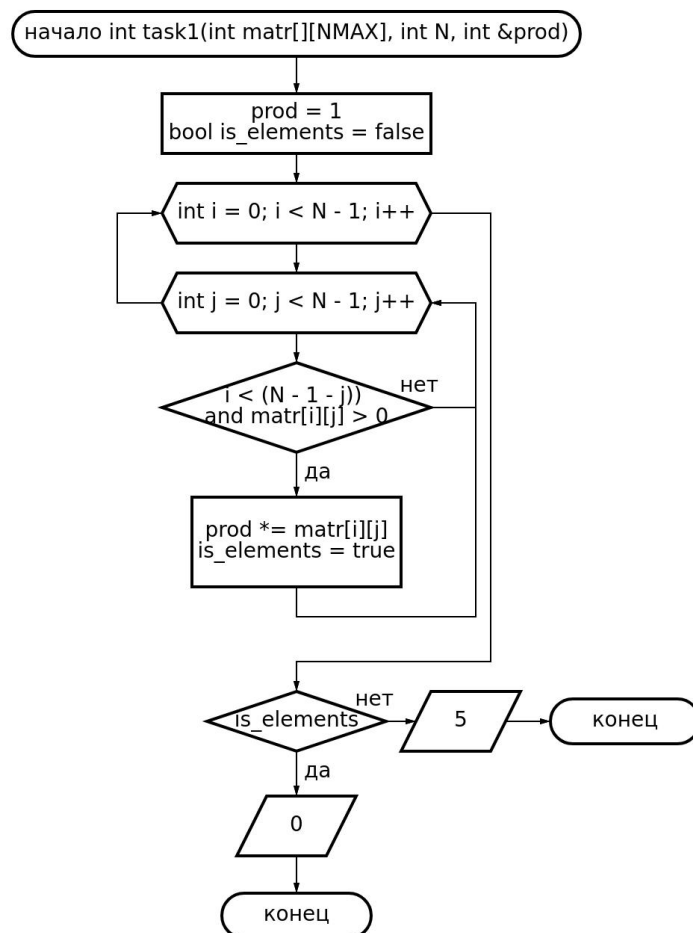
Имя	Тип	Назначение	Вх/Вых
massive	int*	вектор чисел	Вх
flags	bool*	вектор флагов	Вх
max	int	максимум вектора по флагам	Вых

- task1 - функция решения первой задачи

1. Прототип:

```
task1(int matr[][NMAX], int N, int &prod);
```

2. Блок-схема:



3. Описание параметров:

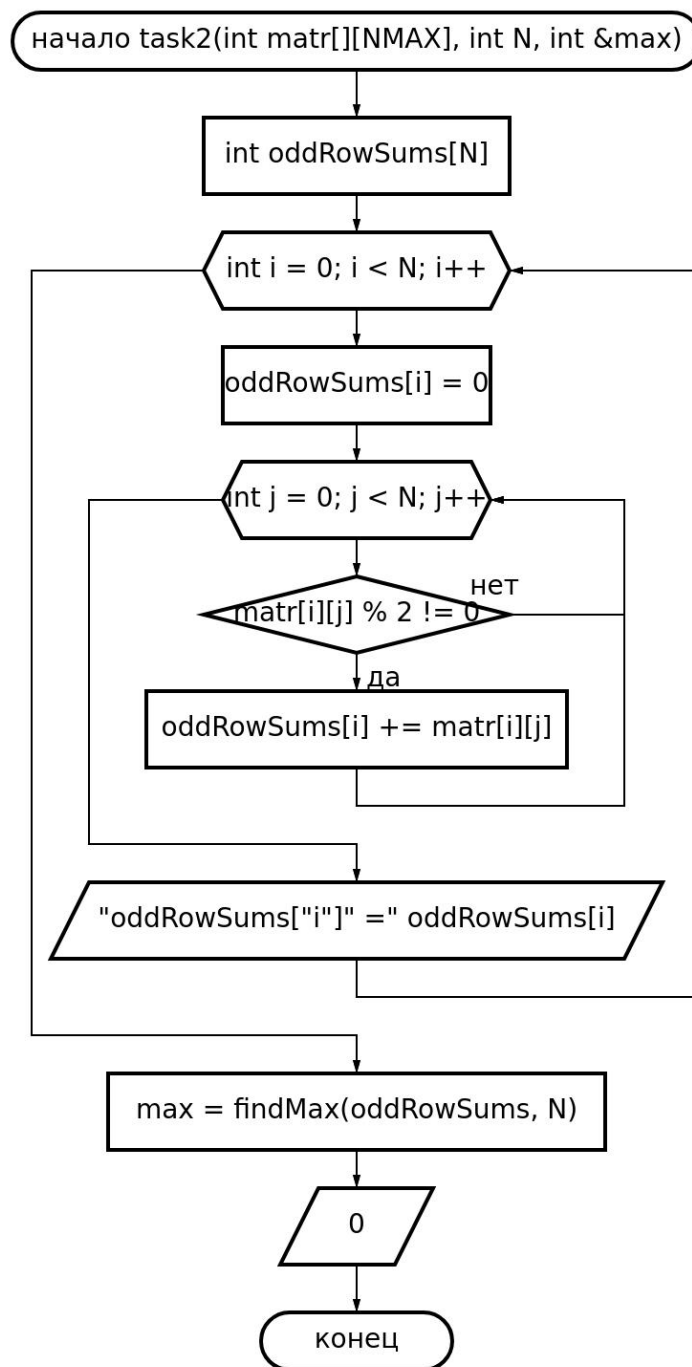
Имя	Тип	Назначение	Вх/Вых
matr	int**	матрица	Вх
N	int	размер матрицы	Вх
prod	&int	результат задачи	Вх/Вых
code	int	код ошибки	Вых

- task2 - функция решения второй задачи

1. Прототип:

```
int task2(int matr[][NMAX], int N, int &max);
```

2. Блок-схема:



3. Описание параметров:

Имя	Тип	Назначение	Вх/Вых
matr	int**	матрица	Вх
N	int	размер матрицы	Вх
max	&int	результат задачи	Вх/Вых
code	int	код ошибки	Вых

Листинг рабочей программы

```
// Creators: Порошин Г. А. , Прошин Е. С.
#include <fstream>
#include <iostream>

using namespace std;

// Объявление констант
const int NMAX = 1000;
// корректные
// const char FNAME1[] = "matr_A.txt";
// const char FNAME2[] = "matr_B.txt";
// const char FNAME1[] = "matr_A1.txt";
// const char FNAME2[] = "matr_B1.txt";
const char FNAME1[] = "matr_A2.txt";
const char FNAME2[] = "matr_B2.txt";
// const char FNAME1[] = "matr_even2.txt";
// const char FNAME2[] = "matr_even3.txt";
// некорректные
// const char FNAME1[] = "nonexistent";
// const char FNAME1[] = "eof.txt";
// const char FNAME1[] = "matr_char.txt";
// const char FNAME1[] = "matr_-2.txt";
// const char FNAME1[] = "matr_A-.txt";
// const char FNAME1[] = "matr_even.txt";
// Заголовки функций
// обработчик ошибок
void error(int code); // код ошибки
// задание матрицы из файла
int defMatr(const char fname[], // название файла
            int matr[][NMAX], // указатель на нулевой элемент квадратной матрицы
            int &N);          // ссылка на размерность матрицы
// очистка матрицы
void delMatr(int matr[][NMAX], // квадратная матрица
             int &N);          // ссылка на размерность матрицы
// печать матрицы
void printMatr(
    const char matr_name[], // название матрицы (А или В)
    int matr[][NMAX],       // указатель на нулевой элемент квадратной матрицы
    int N);                 // размерность матрицы
// печать результата
void printRes(const char out[], // строка, выводимая до печати результата
              int res);         // результат
// первое задание
int task1(int matr[][NMAX], // указатель на нулевой элемент квадратной матрицы
          int N,             // размерность матрицы
          int &prod);        // ссылка на произведение
// второе задание
int task2(int matr[][NMAX], // указатель на нулевой элемент квадратной матрицы
          int N,             // размерность матрицы
          int &max);         // ссылка на максимум
// поиск максимума вектора
int findMax(
    int *massive, // указатель на вектор значений сумм
    bool *flags,  // указатель на вектор с флагами наличия элементов суммы
    int N);       // размерность матрицы
int main() {
    // инициализация переменных
    int matr[NMAX][NMAX]; // квадратная матрица
    int N;                 // размерность матрицы
```

```

int err_code;          // код ошибки
int res;               // результат задания
// решение задач для первой матрицы
err_code = defMatr(FNAME1, matr, N);
error(err_code); // проверяем наличие ошибки
if (!err_code) {
    printMatr("A", matr, N); // печать матрицы
    // решение первой задачи
    err_code = task1(matr, N, res);
    error(err_code);
    if (!err_code) {
        printRes("Задание 1.\n\tПроизведение всех положительных\n\tчисел над "
            "побочной диагональю равно",
            res);
    }
    // решение второй задачи
    err_code = task2(matr, N, res);
    error(err_code);
    if (!err_code) {
        printRes(
            "Задание 2.\n\tМаксимум среди сумм по строкам нечётных\n\tэлементов "
            "матрицы равен",
            res);
        // очистка матрицы
        delMatr(matr, N);
    }
}
// решение задач для второй матрицы
err_code = defMatr(FNAME2, matr, N);
error(err_code); // проверяем наличие ошибки
if (!err_code) {
    printMatr("B", matr, N); // печать матрицы
    // решение первой задачи
    err_code = task1(matr, N, res);
    error(err_code);
    if (!err_code) {
        printRes("Задание 1.\n\tПроизведение всех положительных\n\tчисел над "
            "побочной диагональю равно",
            res);
    }
    // решение второй задачи
    err_code = task2(matr, N, res);
    error(err_code);
    if (!err_code) {
        printRes(
            "Задание 2.\n\tМаксимум среди сумм по строкам нечётных\n\tэлементов "
            "матрицы равен",
            res);
        // очистка матрицы
        delMatr(matr, N);
    }
}
return 0;
}
// Реализации функций
// обработчик ошибок
void error(int code) {
    if (code == 0)
        return;
    cout << "ERROR: ";
}

```

```

switch (code) {
case 1:
    cout << "Не удалось открыть файл!\n";
    break;
case 2:
    cout << "Встречен непредвиденный конец файла!\n";
    break;
case 3:
    cout << "Задан неверный размер матрицы или превышен лимит!\n";
    break;
case 4:
    cout << "Получено не число!\n";
    break;
case 5:
    cout << "Матрица не содержит положительных элементов над побочной "
           "диагональю!\n";
    break;
case 6:
    cout << "Матрица не содержит нечётные элементы!\n";
    break;
}
}
// функция задания матрицы из файла
int defMatr(const char fname[], int matr[][NMAX], int &N) {
    // инициализация переменных
    ifstream fin;
    fin.open(fname);
    cout << "\nDEFMATR: считывание из файла " << fname << "\n";
    // входной контроль
    if (!fin)
        return 1;
    fin >> N;
    if (fin.eof())
        return 2;
    if (fin.fail())
        return 4;
    if (N <= 0 or N > NMAX)
        return 3;
    // задание матрицы из файла
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            fin >> matr[i][j];
            if (fin.eof())
                return 2;
            if (fin.fail())
                return 4;
        }
    }
    fin.close(); // закрытие файла
    return 0;
}
// очистка матрицы
void delMatr(int matr[][NMAX], int &N) {
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            matr[i][j] = 0;
    N = 0;
}
// печать матрицы
void printMatr(const char matr_name[], int matr[][NMAX], int N) {
    cout << "\n" << matr_name << "[" << N << "x" << N << "]=\n";
}

```

```

for (int i = 0; i < N; i++) {
    cout << "\t";
    for (int j = 0; j < N; j++) {
        cout << matr[i][j] << "\t";
    }
    // печать скобок
    if (i == 0)
        cout << "\r\r";
    else if (i == N - 1)
        cout << "\r\r";
    else
        cout << "\r\r";
    cout << "\n";
}
}
// печать результата
void printRes(const char out[], int res) { cout << out << " " << res << "\n"; }
// первое задание: поиск произведения всех
// положительных чисел над побочной диагональю
int task1(int matr[][NMAX], int N, int &prod) {
    // инициализация переменных
    prod = 1;
    bool is_elements = false;
    // поиск элементов и подсчёт произведения
    for (int i = 0; i < N - 1; i++) {
        for (int j = 0; j < N - 1; j++) {
            if ((i < (N - 1 - j)) and matr[i][j] > 0) {
                prod *= matr[i][j];
                is_elements = true;
            }
        }
    }
    // проверка на положительные элементы
    if (!is_elements)
        return 5;
    return 0;
}
// поиск максимума вектора
int findMax(int *massive, bool *flags, int N) {
    // инициализация переменных
    int maxi;
    int max;
    for (int i = 0; i < N; i++) { // ищем первую валидную сумму
        if (flags[i]) {
            maxi = i;
            max = massive[i];
            break;
        }
    }
    // поиск максимума
    for (int i = 0; i < N; i++) {
        if (massive[i] > max and flags[i]) {
            max = massive[i];
            maxi = i;
        }
    }
    return max;
}
// второе задание: поиск максимума среди сумм
// по строкам нечётных элементов матрицы
int task2(int matr[][NMAX], int N, int &max) {

```

```

// инициализация переменных
int oddRowSums[N];
bool oddRowFlag[N];
for (int i = 0; i < N; i++)
    oddRowFlag[i] = false;
bool is_elements = false;
// посчёт и вывод сумм
for (int i = 0; i < N; i++) {
    oddRowSums[i] = 0;
    for (int j = 0; j < N; j++) {
        if (matr[i][j] % 2 != 0) {
            oddRowSums[i] += matr[i][j];
            oddRowFlag[i] = true;
            is_elements = true;
        }
    }
    // эхо-печать
    if (oddRowFlag[i])
        cout << "TASK2: oddRowSums[" << i << "]=" << oddRowSums[i] << "\n";
}
// проверка на наличие нечётных элементов
if (!is_elements)
    return 6;
// поиск максимума
max = findMax(oddRowSums, oddRowFlag, N);
return 0;
}

```


Тесты

Некорректные тесты

1.

Цель: проверить работу программы при отсутствии файла

Исходные данные: -

Ожидаемый результат: Не удалось открыть файл!

Результат:

Не удалось открыть файл!

Вывод: полученное значение совпало с ожидаемым. Тест ошибку не обнаружил.

2.

Цель: проверить работу программы при пустом файле

Исходные данные: -

Ожидаемый результат: Встречен непредвиденный конец файла!

Результат:

Встречен непредвиденный конец файла!

Вывод: полученное значение совпало с ожидаемым. Тест ошибку не обнаружил.

3.

Цель: проверить работу программы при неверном чтении

Исходные данные: a 1 1 1 1

Ожидаемый результат: Получено не число!

Результат:

Получено не число!

Вывод: полученное значение совпало с ожидаемым. Тест ошибку не обнаружил.

4.

Цель: проверить работу программы при неправильном размере матрицы

Исходные данные: -2 1 1 1 1

Ожидаемый результат: Задан неверный размер матрицы!

Результат:

Задан неверный размер матрицы!

Вывод: полученное значение совпало с ожидаемым. Тест ошибку не обнаружил.

5.

Цель: проверить работу программы при отсутствии положительных чисел над побочной диагональю

Исходные данные: 2 -1 1 1 1

Ожидаемый результат: Матрица не содержит положительных элементов над побочной диагональю!

Результат:

Матрица не содержит положительных элементов над побочной диагональю!

Вывод: полученное значение совпало с ожидаемым. Тест ошибку не обнаружил.

6.

Цель: проверить работу программы при отсутствии нечётных элементов

Исходные данные: 3 2 4 6 8 2 4 6 8 2

Ожидаемый результат: Матрица не содержит нечётные элементы!

Результат:

Матрица не содержит нечётные элементы!

Вывод: полученное значение совпало с ожидаемым. Тест ошибку не обнаружил.

Корректные тесты

1.

Цель: проверить работу программы при корректных данных

Исходные данные:

```
4
1 2 -3 2
-5 -6 1 1
-9 1 1 1
1 1 1 1
```

```
5
2 2 2 2 1
2 2 2 1 1
2 2 1 1 1
2 1 1 1 1
1 1 1 1 1
```

Ожидаемый результат:

DEFMATR: считывание из файла matr_A2.txt

A[4x4]=

$$\begin{bmatrix} 1 & 2 & -3 & 2 \\ -5 & -6 & 1 & 1 \\ -9 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Задание 1.

Произведение всех положительных чисел над побочной диагональю равно 2

TASK2: oddRowSums[0]=-2

TASK2: oddRowSums[1]=-3

TASK2: oddRowSums[2]=-6

TASK2: oddRowSums[3]=4

Задание 2.

Максимум среди сумм по строкам нечётных элементов матрицы равен 4

DEFMATR: считывание из файла matr_B2.txt

B[5x5]=

$$\begin{bmatrix} 2 & 2 & 2 & 2 & 1 \\ 2 & 2 & 2 & 1 & 1 \\ 2 & 2 & 1 & 1 & 1 \\ 2 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Задание 1.

Произведение всех положительных чисел над побочной диагональю равно 1024

TASK2: oddRowSums[0]=1

TASK2: oddRowSums[1]=2

TASK2: oddRowSums[2]=3

TASK2: oddRowSums[3]=4

TASK2: oddRowSums[4]=5

Задание 2.

Максимум среди сумм по строкам нечётных элементов матрицы равен 5

Результат:

```

DEFMATR: считывание из файла matr_A2.txt

A[4x4]=
[
    1      2      -3      2
    -5     -6      1      1
    -9      1      1      1
    1       1      1      1
]

Задание 1.
    Произведение всех положительных
    чисел над побочной диагональю равно 2
TASK2: oddRowSums[0]=-2
TASK2: oddRowSums[1]=-3
TASK2: oddRowSums[2]=-6
TASK2: oddRowSums[3]=4
Задание 2.
    Максимум среди сумм по строкам нечётных
    элементов матрицы равен 4

DEFMATR: считывание из файла matr_B2.txt

B[5x5]=
[
    2      2      2      2      1
    2      2      2      1      1
    2      2      1      1      1
    2      1      1      1      1
    1      1      1      1      1
]

Задание 1.
    Произведение всех положительных
    чисел над побочной диагональю равно 1024
TASK2: oddRowSums[0]=1
TASK2: oddRowSums[1]=2
TASK2: oddRowSums[2]=3
TASK2: oddRowSums[3]=4
TASK2: oddRowSums[4]=5
Задание 2.
    Максимум среди сумм по строкам нечётных
    элементов матрицы равен 5

```

Вывод: полученное значение совпало с ожидаемым. Тест ошибку не обнаружил.

2.

Цель: проверить работу программы при большем количестве пробелов в строке

Исходные данные:

```

4
1 2 2 2
2 -6 1 1
-9 1 1 1
1 2 1 1

```

```

5
4 2 1 2 1
2 1 2 1 1
1 2 1 1 1
2 1 1 1 1
1 1 1 1 1

```

Ожидаемый результат:

```
DEFMATR: считывание из файла matr_A3.txt
```

```

A[4x4]=
[
    1      2      2      2
    2     -6      1      1
    -9      1      1      1
    1       2      1      1
]

```

```

Задание 1.
    Произведение всех положительных
    чисел над побочной диагональю равно 8
TASK2: oddRowSums[0]=1
TASK2: oddRowSums[1]=2

```

TASK2: oddRowSums[2]=-6
 TASK2: oddRowSums[3]=3
 Задание 2.
 Максимум среди сумм по строкам нечётных
 элементов матрицы равен 3

DEFMATR: считывание из файла matr_B3.txt

B[5x5]=

$$\begin{bmatrix} 4 & 2 & 1 & 2 & 1 \\ 2 & 1 & 2 & 1 & 1 \\ 1 & 2 & 1 & 1 & 1 \\ 2 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Задание 1.
 Произведение всех положительных
 чисел над побочной диагональю равно 256

TASK2: oddRowSums[0]=2
 TASK2: oddRowSums[1]=3
 TASK2: oddRowSums[2]=4
 TASK2: oddRowSums[3]=4
 TASK2: oddRowSums[4]=5

Задание 2.
 Максимум среди сумм по строкам нечётных
 элементов матрицы равен 5

Результат:

DEFMATR: считывание из файла matr_A3.txt

A[4x4]=

$$\begin{bmatrix} 1 & 2 & 2 & 2 \\ 2 & -6 & 1 & 1 \\ -9 & 1 & 1 & 1 \\ 1 & 2 & 1 & 1 \end{bmatrix}$$

Задание 1.
 Произведение всех положительных
 чисел над побочной диагональю равно 8

TASK2: oddRowSums[0]=1
 TASK2: oddRowSums[1]=2
 TASK2: oddRowSums[2]=-6
 TASK2: oddRowSums[3]=3

Задание 2.
 Максимум среди сумм по строкам нечётных
 элементов матрицы равен 3

DEFMATR: считывание из файла matr_B3.txt

B[5x5]=

$$\begin{bmatrix} 4 & 2 & 1 & 2 & 1 \\ 2 & 1 & 2 & 1 & 1 \\ 1 & 2 & 1 & 1 & 1 \\ 2 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Задание 1.
 Произведение всех положительных
 чисел над побочной диагональю равно 256

TASK2: oddRowSums[0]=2
 TASK2: oddRowSums[1]=3
 TASK2: oddRowSums[2]=4
 TASK2: oddRowSums[3]=4
 TASK2: oddRowSums[4]=5

Задание 2.
 Максимум среди сумм по строкам нечётных
 элементов матрицы равен 5

Вывод: полученное значение совпало с ожидаемым. Тест ошибку
 не обнаружил.

Вывод

Разработка программы завершена на том основании, что:

- полученные результаты совпали с ожидаемыми
- считаем набор тестов полным